

Proyecto Programado CI-0123
Armador de figuritas con piezas plásticas (Lego®)
Autores: Equipo 4

Socios: Equipo 1 (BST)
Equipo 2 (Los teletubbies)
Equipo 3 (Brigada A)
Plan B

Protocolo Grupal-Uppercase v2.3

En este documento se explica el protocolo de comunicación entre los elementos del proyecto. Se tiene un cliente, un intermediario (fork) y un servidor. El cliente únicamente se comunica con el fork, quien a su vez se comunica con el servidor. El servidor se comunica con el fork, que transmite sus mensajes al cliente. Varios clientes y servidores se pueden conectar al mismo fork, por lo que este crea un hilo para manejar independientemente cada conexión ($C_1 - F - S_1$).

1. Descripción

El protocolo tiene 3 parámetros separados por el carácter */*. Se llama “Uppercase” ya que sus dos primeros parámetros sólo utilizan una letra mayúscula para indicar el tipo de mensaje a transmitir. Los mensajes siguen una estructura estándar con el siguiente formato:

[Protocolo]/[Comando]/[mensaje]

Dónde *Protocolo* siempre es la letra P, su única función es indicar que se está utilizando este protocolo. *Comando* es otra letra mayúscula que indica la función a realizar (véase Tabla 1). El tercer parámetro es un mensaje opcional, se utiliza para enviar datos (directorios y figuras) o información (puerto y dirección del servidor). En algunos casos, como en un cierre de conexión, *mensaje* puede omitirse, ya que el comando por sí solo describe completamente la acción.

Como se tiene la restricción de ser 3 parámetros y de que los dos primeros sean solo una letra entonces se sabe que los primeros 4 caracteres son parte del protocolo y el resto es parte del mensaje. Esto permite que se pueda utilizar “P/” dentro del mensaje sin generar problemas.

Ventajas:

- Implementación: la construcción, procesamiento y limpieza del protocolo conllevan métodos fáciles de realizar para el programador de los elementos.
- User Friendly: por su sencillez, cualquier usuario puede aprender y utilizarlo rápidamente.

Desventajas:

- Escalabilidad: limita a un máximo de 26 comandos (26 letras en el abecedario). No se pueden

agregar más parámetros sin romper el formato, hacerlo implicaría tener que considerar casos particulares en los mensajes. Solo se puede enviar un comando a la vez.

2. Comandos

A continuación se muestra una tabla con los comandos y una breve descripción.

Nombre	Representación	Descripción
ACK	A	Indica que el comando se hizo correctamente
Connect	C	Crear conexión inicial fork-server
Data	D	Respuesta del server (directorios, figuras)
Get	G	Solicitud de figuras
Quit	Q	Cerrar conexión fork-server
Request	R	Solicitud de servidores o directorios
Server Data	S	Dirección IP y puerto del servidor

Tabla 1. Comandos del Protocolo

3. Flujo de Conexión

Descubrimiento de Servidores

Primero se levanta el intermediario, este crea un proceso secundario para realizar broadcast a la dirección 172.16.123.79 preguntando por servidores disponibles con el siguiente mensaje:

Fork > P/C/172.16.123.70:8080 > Broadcast*

Cuando se levanta un servidor escucha por broadcast, recibirá el mensaje anterior con lo que obtendrá la dirección y puerto del intermediario. Ahora el servidor establece una conexión TCP con el intermediario y le envía sus propios datos con el siguiente mensaje:

Server > P/S/172.16.123.71:8080 > Fork

El intermediario crea otro proceso secundario para atender cada conexión TCP. Recibe los datos del servidor y los agrega en una estructura de datos compartida entre el resto de procesos para que funcione como mapa de servidores. El servidor cierra la conexión TCP que tiene con el intermediario con un comando QUIT para que ambos se salgan de sus respectivos ciclos.

(Nota* si el servidor recibe el mensaje de broadcast con *recvFrom()* ya obtiene la dirección y puerto del intermediario en *addr*, por lo que no es necesario enviar estos datos)

Comunicación con Cliente

El cliente conoce la dirección IP y puerto del intermediario, cuando inicia establece una conexión TCP. El cliente puede digitar comandos siguiendo el protocolo establecido, si digita otra cosa el programa le indicará que cometió un error. El cliente puede solicitar el directorio de figuras (REQUEST), una figura en particular (GET) o terminar la conexión (QUIT).

Siguiendo un flujo tradicional, primero solicita el directorio de figuras disponibles. El intermediario hace una solicitud del directorio a cada servidor que tenga disponible y junta todas las posibles figuras en una lista* que le será enviada al cliente. Usa los siguientes mensajes:

```
// Solicitud a intermediario
Client > P/R/dir > Fork
// Para cada servidor
Fork > P/R/dir > Server
Server > P/D/fig1, fig2, fig3 > Fork
// Respuesta a cliente
Fork > P/D/fig1, fig2, fig3 > Client
```

Una vez tiene el listado de figuras, el cliente puede solicitar alguna. El intermediario recibe la solicitud, como tiene un mapa de servidores con figuras lo utiliza para identificar el servidor que tiene esta figura, establece una conexión TCP solo con ese servidor y reenvía la solicitud. Si durante este tiempo el servidor actualizó sus figuras y ahora no cuenta con la que fue solicitada responderá con un mensaje de error. De lo contrario responde con la figura. Intercambio:

```
// Solicitud a intermediario
Client > P/G/fig1 > Fork
// Envía solicitud a servidor
Fork > P/G/fig1 > Server
Server > P/D/brown block 1, red block... > Fork
// Respuesta a cliente
Fork > P/D/brown block 1, red block > Client
```

El cliente recibe la figura, vuelve al menú. Para pedir otra figura u otra vez el directorio disponible repite el proceso anterior. Para terminar la conexión utiliza el comando de QUIT, cierra el socket conectado al intermediario y termina el programa. Este comando también se utiliza para cerrar las conexiones TCP entre intermediario y servidor cada vez que se hace un intercambio de mensajes entre ellos.

```
Client > P/Q/ > Fork
```

(Nota* en caso de que se repita el nombre de una figura entre servidores, el intermediario debe crear un identificador para que el cliente pueda diferenciarlas)

Comunicación con Cliente Web

Todos los elementos (Client, Fork, Server) tienen la capacidad de codificar y decodificar el protocolo de comunicación. A la hora de decodificar un mensaje HTTP el programa detecta que no se está utilizando el protocolo e inicia un procedimiento para manejar HTTP.

En caso de que el servidor reciba el mensaje, extrae la solicitud con lo que obtiene un nombre de figura. La lee de su sistema de archivos y la agrega a una respuesta que sigue de manera sencilla el protocolo HTTP. Le responde al elemento que haya realizado la solicitud y cierra la conexión.

En caso de que sea el intermediario el que recibe la solicitud, de igual forma detecta que no se está siguiendo el protocolo, extrae el nombre de la figura para saber a qué servidor debe solicitarla, establece una conexión TCP y le envía al mensaje tal cual. El servidor realiza el proceso descrito anteriormente, responde con un mensaje HTTP al intermediario. El intermediario recibe el mensaje y lo transmite al cliente web.

Notas Generales:

- Las direcciones IPv4 utilizadas en este ejemplo de flujo de conexión son las asignadas al equipo 4 del grupo 3 del laboratorio 3-5.
- De momento no se utiliza el comando ACK durante el flujo de conexión, pero lo dejamos como auxiliar en caso de que se presente un error a futuro.

Direcciones DHCP asignables por isla BC (Dirección de Broadcast)

- Isla 1: 172.16.123.[19-30] > BC 172.16.123.31
- Isla 2: 172.16.123.[35-46] > BC 172.16.123.47
- Isla 3: 172.16.123.[51-62] > BC 172.16.123.63
- Isla 4: 172.16.123.[67-78] > BC 172.16.123.79
- Isla 5: 172.16.123.[83-94] > BC 172.16.123.95
- Isla 6: 172.16.123.[99-110] > BC 172.16.123.111

Mapa de las VLAN/IP

- Isla 1: VLAN 610, IP 172.16.123.16/28, IP L3 .17, IP L2 .18
- Isla 2: VLAN 620, IP 172.16.123.32/28, IP L3 .33, IP L2 .34
- Isla 3: VLAN 630, IP 172.16.123.48/28, IP L3 .49, IP L2 .50
- Isla 4: VLAN 640, IP 172.16.123.64/28, IP L3 .65, IP L2 .66
- Isla 5: VLAN 650, IP 172.16.123.80/28, IP L3 .81, IP L2 .82
- Isla 6: VLAN 660, IP 172.16.123.96/28, IP L3 .97, IP L2 .98

4. Posibles Eventos-Bitácoras

Server se levanta antes que el Fork: Server se quedará esperando por el mensaje en Broadcast.

Client se levanta antes que el Fork: a la hora de intentar establecer la conexión con el Fork el socket reporta error y se termina el programa.

Client se levanta antes que algún Server: cuando el cliente le pide al Fork la lista de servidores disponibles este le responde con “P/S/” lo que le indica que todavía no tiene ninguno.

Client solicita una figura que no existe: Fork responde con un mensaje de error 404.

Client solicita una figura que ha sido borrada: Server responde con un mensaje de error 404.

Client utiliza REQUEST incorrectamente: el comando de REQUEST sirve para pedir el directorio de figuras de los servidores disponibles. Si solicita otra cosa, por ejemplo una figura, el servidor le responderá con una advertencia.

Server se cierra durante la conexión: cuando el cliente envía una solicitud, el Fork la recibe, detecta que el Server ya no está y le responde al cliente “P/D/”. El cliente por debajo envía un QUIT para que se termine la conexión y luego puede establecer otra con un Server activo.

Fork se cierra durante la conexión: cuando el cliente envía una solicitud el socket reporta error y el programa termina.

Client se cierra sin hacer QUIT: el cliente termina de golpe, por ejemplo con un ctrl+C, se contemplan 2 posibles alternativas de solución a desarrollar:

1. Hacer que el programa detecte si termina de golpe y enviar un QUIT automático.
2. Tanto Server como Fork tienen un límite de tiempo de espera.

TODO establecer reglas o comandos para:

"Muerte" de servidores

- Método de finalización
- Mensajes a servidores propios relacionados
- Mensajes a otras islas, los servidores deben anunciar su "muerte" a los otros interesados

Descubrimiento de servidores intermediarios (I) y de figuras (F)

- En otras islas: I a I